StateSet STARK: Security Advisory

Confidentiality claims withdrawn — 2026-09-05

This PDF is a historical whitepaper, not a description of supported guarantees.
The current proof transcript can disclose transaction amounts and witness salts.
Salted commitments do not make the proof transcript zero knowledge.

Claims below about private computation or production readiness are withdrawn.
Confidentiality requirements now fail closed through privacy-aware APIs.
Explicit disclosed workflows support computational integrity only.

Current implementation and release gates:
docs/SECURITY_ADVISORY_AMOUNT_DISCLOSURE.md
docs/REFUND_PILOT.md

Independent cryptographic review and a reviewed zero-knowledge construction
are still required before a confidential release. Historical proofs cannot be
made confidential retroactively.

STATESET — TECHNICAL WHITEPAPER · 2026

Cryptographic Compliance for *Commerce*

A production-grade STARK system for proving regulatory compliance without revealing transaction amounts

VERSION	CLASSIFICATION	PROTOCOL	PUBLISHED
0.2.1	Public	VES-STARK	March 2026

EXECUTIVE SUMMARY

StateSet STARK is a transparent, post-quantum secure zero-knowledge proof system that enables commerce platforms to demonstrate regulatory compliance—anti-money laundering thresholds, order caps, delegated authorization limits—without ever exposing the underlying transaction amounts. Built on the Winterfell STARK framework with the Goldilocks prime field and Rescue-Prime hashing, the system produces verifiable proofs in under two seconds and verifies them in under 200 milliseconds, with no trusted setup required.

CONTENTS

Section I

The Compliance Paradox

Every compliance check in digital commerce faces the same contradiction: regulators need proof that a transaction satisfies their rules, but merchants and consumers need those transaction amounts to remain private.

Anti-money laundering regulations require platforms to verify that individual transactions fall below reporting thresholds. Order management systems must enforce caps set by merchants and payment processors. Delegated commerce agents must prove they operate within authorized limits. In every case, the verifier needs a guarantee about the *relationship* between a value and a constraint—not the value itself.

Today, this is solved by disclosure. Transaction amounts flow in plaintext to compliance engines, payment processors, audit logs, and third-party services. Each disclosure point creates a new attack surface, a new liability under GDPR and CCPA, and a new vector for competitive intelligence leakage. The industry accepts this because, until now, there was no practical alternative.

> *The question is not whether an amount satisfies a threshold. The question is whether you can prove it does—without revealing the amount at all.*

StateSet STARK provides the alternative: a cryptographic proof system purpose-built for commerce compliance. A prover generates a mathematical certificate—a STARK proof—that a private amount satisfies a policy constraint. The proof can be verified by anyone, in milliseconds, without learning anything about the amount beyond the

THE SCALE OF EXPOSURE

A single mid-market e-commerce platform processes millions of transactions per month, each requiring compliance verification. Every plaintext amount that passes through a compliance pipeline is a data point that must be secured, retained per regulation, and disclosed in the event of a breach. Zero-knowledge proofs eliminate this exposure at the protocol level.

REGULATORY LANDSCAPE

The EU's 6AMLD, the US Bank Secrecy Act, and PSD2 all require compliance verification for transaction amounts. Privacy regulations (GDPR Art. 25, CCPA) simultaneously require data minimization. ZK proofs satisfy both requirements simultaneously.

fact that it complies. No trusted third parties. No key ceremonies. No quantum vulnerabilities.

Section II

A Zero-Knowledge Approach

STARKs—Scalable Transparent Arguments of Knowledge—provide the strongest foundation for commerce compliance verification: no trusted setup, post-quantum security, and transparent verification that anyone can audit.

Unlike SNARKs, which require a one-time trusted setup ceremony that, if compromised, undermines all subsequent proofs, STARKs derive their security entirely from collision-resistant hash functions and the algebraic structure of finite fields. There is no trapdoor, no toxic waste, no master key that could be stolen or subpoenaed.

This transparency is not merely a theoretical advantage. In a compliance context, the absence of trusted setup means that no party—not the prover, not the platform, not a government agency—possesses a secret that could be used to forge compliant-looking proofs for non-compliant transactions. The integrity of the system rests on public mathematics, not on the operational security of a key ceremony.

Why STARKs for Commerce

T

Transparency

No trusted setup eliminates the single point of failure that compliance auditors rightly distrust. Verification logic is publicly inspectable.

Q

Post-Quantum Security

STARK security relies on hash functions, not elliptic curve discrete logarithms. Compliance records that must remain valid for decades are protected against quantum advances.

S

Scalable Verification

Verification cost grows logarithmically with computation size, making batch proofs over hundreds of transactions economically viable.

A

Algebraic Soundness

The constraint system is formally auditable. Each constraint maps directly to a compliance requirement, enabling mathematical verification of the proof system itself.

$$\exists\, amount \in [0, 2^{64}) \;:\; amount < threshold \;\wedge\; C = Rescue(amount) \;\wedge\; P \text{ binds public inputs}$$

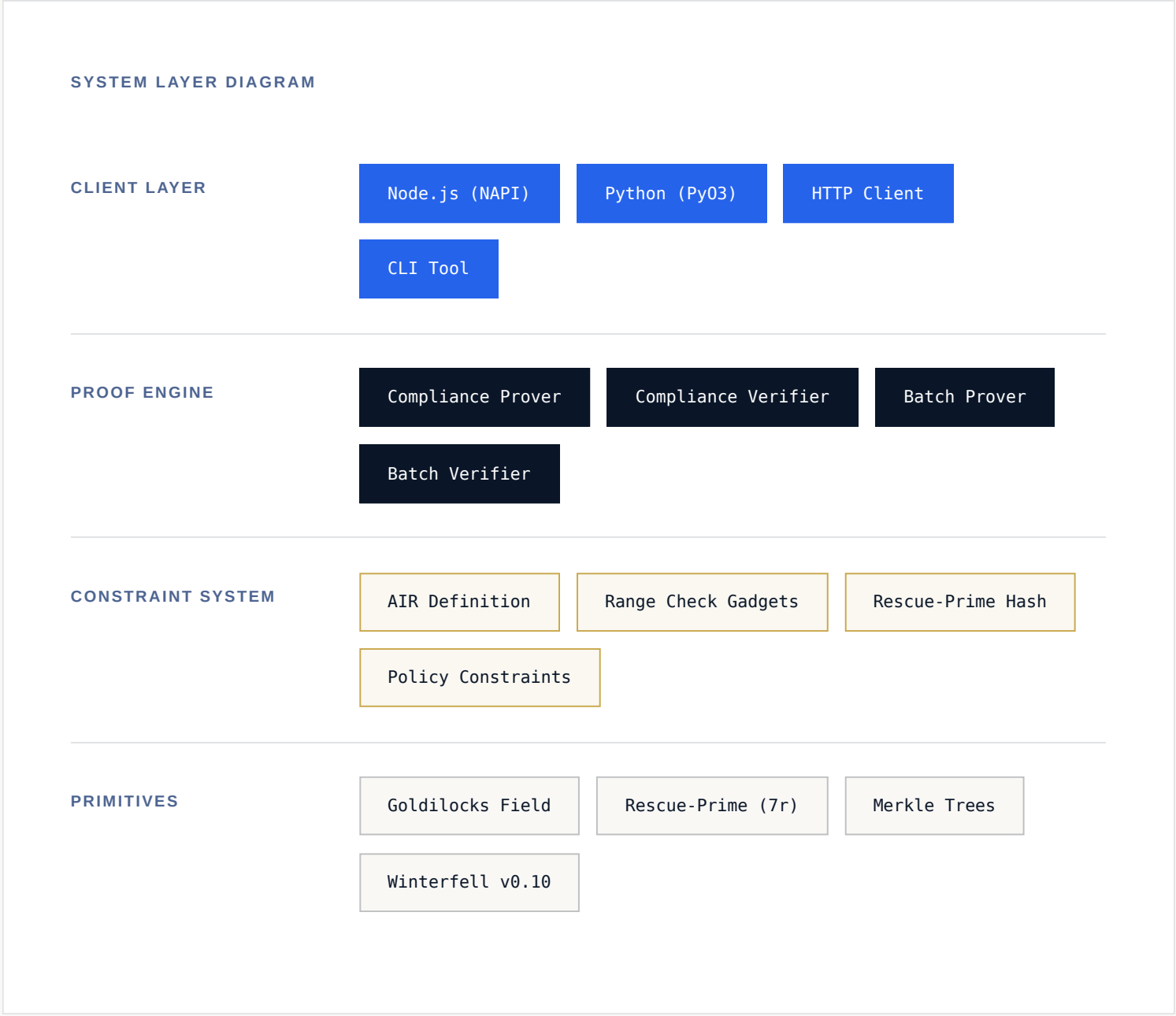
The core proof statement: existence of a compliant amount, cryptographically committed and policy-bound.

<2s	<200ms	~150KB	128+
PROOF GENERATION	VERIFICATION TIME	PROOF SIZE	BATCH CAPACITY

Section III

System Architecture

The VES-STARK system is organized as a modular Rust workspace of nine crates, each with a clearly defined responsibility. This separation enables independent compilation, testing, and deployment of each layer.



Crate Topology

The workspace root contains nine crates organized by function. `ves-stark-primitives` provides the field arithmetic, Rescue hash implementation, and canonical data structures that every other crate depends on. The `air` crate defines the Algebraic Intermediate Representation—the constraint system that encodes compliance logic as polynomial equations over the Goldilocks field.

The `prover` and `verifier` crates handle single-event compliance proofs. The `batch` crate extends this to multi-event state transitions with Merkle accumulation. The client-facing crates— `client` ,

Design Principles

Minimal surface area. Each crate exports only what downstream consumers need. The constraint system is never exposed to client code; only proof bytes and verification results cross the API boundary.

Zero-copy verification. The verifier operates on serialized proof bytes without requiring the prover's state, witness, or trace data. This enables verification in constrained environments—edge nodes, smart contracts, or browser-based validators.

Deterministic canonicalization. Public inputs are serialized using RFC 8785 JSON Canonicalization

`cli`, `nodejs`, and `python` —provide integration surfaces for different deployment contexts.

Scheme, ensuring that prover and verifier always agree on the exact bytes being proven, regardless of JSON library or platform.

The proof does not reveal the amount. It does not reveal the distance from the threshold. It reveals one bit of information: the transaction complies.

Section IV

Cryptographic Foundations

The Goldilocks Field

All arithmetic in VES-STARK operates over the Goldilocks prime field with modulus $p = 2^{64} - 2^{32} + 1$. This "just right" prime provides native 64-bit arithmetic on modern CPUs while supporting the large multiplicative subgroups required for efficient FFT-based polynomial operations.

Field elements fit in a single machine register, eliminating the multi-precision arithmetic overhead that plagues pairing-based systems like BLS12-381. Modular reduction uses only shifts and additions—no division. The result is proof generation that runs at near-native speed without specialized hardware.

FIELD PARAMETERS

Prime modulus	$2^{64} - 2^{32} + 1$
Decimal value	`18446744069414584321`
Word size	64 bits (native)
Largest 2-adic subgroup	2^{32}
Generator	7

Rescue-Prime Hash

Witness commitments use the Rescue-Prime hash function, a STARK-friendly algebraic hash designed for efficient in-circuit evaluation. Unlike SHA-256 or Blake3, whose bitwise operations are expensive to express as field constraints, Rescue-Prime uses a power-map S-box (x^7) that maps directly to low-degree polynomial constraints.

The implementation uses 7 rounds with a 12-element state (8-element rate, 4-element capacity) in sponge mode. This produces a 4-element commitment (256 bits of security in the Goldilocks field) from the 8-limb decomposition of a 64-bit witness amount.

RESCUE-PRIME CONFIGURATION

Rounds	7 half-rounds
State width	12 field elements
Rate / Capacity	8 / 4 elements
S-box	x^7 (forward), x^7 (backward)
Commitment output	4 field elements

Algebraic Intermediate Representation

The AIR encodes the compliance proof as a system of polynomial constraints over an execution trace. Each row of the trace represents a step of the computation; each column holds a register value. Transition constraints enforce that successive rows follow the correct computation, while boundary constraints pin specific values at known positions.

The per-event AIR uses **96 trace columns** across a minimum of **128 rows** (power-of-two). The trace encodes the witness amount decomposition, threshold limbs, subtraction carry chain, binary bit decomposition for range checking, Rescue-Prime state evolution, and public input binding values.

The constraint system comprises **157 transition constraints** and **80 boundary assertions**. The maximum constraint degree is 10, arising from the Rescue x^7 S-box multiplied by a row selector. This degree determines the minimum LDE blowup factor of 16×, balancing proof size against soundness.

CONSTRAINT DEGREE GROUPS

Degree 2: Structural constraints—binary checks, round counters, flag selectors.

Degree 3–4: Input/output binding—connecting witness limbs to hash inputs and public input fields.

Degree 10: Rescue half-round transitions—the x^7 S-box under a periodic row selector.

Degree declarations use Winterfell's `TransitionConstraintDegree` API with periodic column cycle annotations for accurate quotient computation.

Section V

Multi-Policy Framework

The system supports three distinct compliance policy types, each encoding a different regulatory or business constraint as an algebraic inequality within the same AIR framework.

`aml.threshold` 01

AML Threshold

Proves that a transaction amount falls strictly below a regulatory reporting threshold. The constraint enforces `amount < threshold` via limb-wise subtraction with borrow propagation.

Applicable to: CTR reporting limits, cross-border transaction caps, suspicious activity thresholds.

`order_total.cap` 02

Order Cap

Proves that an order total does not exceed a merchant- or platform-defined maximum. The constraint enforces `amount ≤ cap`, differing from AML in its inclusive upper bound.

Applicable to: Merchant order limits, payment processor caps, marketplace spending controls.

`agent.authorization.v1` 03

Agent Authorization

Proves that a delegated commerce agent's transaction does not exceed its authorization limit, with an additional binding to an intent hash. Enforces `amount ≤ maxTotal` with intent commitment.

Applicable to: AI shopping agents, procurement automation, delegated purchasing authority.

Policy Hash Binding

Each proof cryptographically binds to its policy via a deterministic hash of the policy identifier and parameters. The verifier independently recomputes this hash from the declared policy and rejects any proof where the binding does not match. This prevents policy substitution attacks—a proof generated against a $10,000 AML threshold cannot be replayed against a $1,000 threshold.

Batch State Transitions

While per-event proofs demonstrate individual compliance, production systems process transactions in batches. The batch proving system aggregates up to 128 events into a single STARK proof with Merkle-based state accumulation.

Each batch proof attests to the following properties simultaneously:

- **Per-event compliance:** Every event in the batch satisfies the shared policy constraint
- **Sequence continuity:** Events appear in the declared order
- **State accumulation:** A Merkle tree over per-event commitment hashes produces the declared state root
- **Transition integrity:** The previous state root and new state root form a valid state transition

Merkle tree construction uses in-circuit Rescue-Prime hashing. Each leaf is the canonical hash of an event's public inputs. Internal nodes combine child hashes through the Rescue sponge. The root is boundary-asserted as a public output, enabling on-chain anchoring with a single 4-element state commitment.

BATCH AIR DIMENSIONS

Transition constraints	636
Boundary assertions	145
Rows per Merkle node	15
Max events per batch	128
Finalize rows	15
Max constraint degree	10

On-Chain Efficiency

A batch proof covering 128 events produces the same ~150 KB proof as a single-event proof. For a blockchain sequencer, this represents a 128× reduction in per-event proof storage and verification cost. The state root transition (prev_root → new_root) is the only value that needs to be anchored on-chain.

End-to-End Batch Flow

01

Accumulate

Events arrive from the sequencer. Each event carries its canonical public inputs, encrypted payload hashes, and policy identifier.

02

Witness

The batch witness builder combines private amounts with public inputs, constructing the execution trace and Merkle tree in memory.

03

Prove

The batch prover generates a single STARK proof over all events. In-circuit Rescue computes Merkle nodes; compliance constraints verify each event.

04

Verify

The batch verifier checks the proof, confirms the state root transition, and returns a composite result covering all events in the batch.

05

Ancho

The ne
ancho
blockc
seque
provid
comm
batch's
status

Section VII

Integration & Developer Experience

The system provides native bindings for the three most common integration contexts in commerce infrastructure: server-side JavaScript, Python data pipelines, and systems-level Rust.

Node.js

Native NAPI bindings compiled for Linux (x64, ARM64), macOS, and Windows. Proof bytes are exposed as `Buffer` objects; witness commitments as hex strings for JSON-safe transport. BigInt support for amounts and thresholds.

JAVASCRIPT

```
const { prove, verify } =
require('@stateset/ves-stark'); const proof
= prove( 5000n, // private amount
publicInputs, // canonical public inputs
'aml.threshold', // policy 10000n //
threshold ); const result = verify(
proof.proofBytes, publicInputs,
proof.witnessCommitmentHex ); // { valid:
true, verificationTimeMs: 142 }
```

Python

PyO3/Maturin bindings for Python ≥ 3.8. Designed for integration with compliance analytics pipelines, audit tooling, and data science workflows.

PYTHON

```
import ves_stark policy =
ves_stark.Policy.aml_threshold(10000) proof
= ves_stark.prove( 5000, # private amount
public_inputs, # canonical inputs policy )
result = ves_stark.verify(
proof.proof_bytes, public_inputs,
proof.witness_commitment ) # result.valid
== True # result.verification_time_ms ~ 150
```

CLI Tool

A standalone binary for scripting, CI/CD integration, and operational tooling. Supports proof generation, verification, batch operations, and test data generation.

```shell
# Generate a compliance proof $ ves-stark prove \ --amount 5000 \ --limit 10000 \ --policy aml.threshold \ --inputs public_inputs.json \ --output proof.b64 # Verify $ ves-stark verify \ --proof proof.b64 \ --inputs public_inputs.json Proof valid. Verified in 147ms.
```

Sequencer Client

The Rust HTTP client integrates directly with the StateSet sequencer and Set Chain for production proof submission and state anchoring.

```rust
let client = SequencerClient::try_new( "https://seq.stateset.io", api_key )?; // Fetch & validate public inputs let inputs = client .get_public_inputs_validated( event_id, "aml.threshold", 10000 ).await?; // Submit proof to registry client.submit_proof(submission).await?;
```

Security Model & Threat Analysis

The security of VES-STARK rests on three pillars: the algebraic soundness of the constraint system, the collision resistance of Rescue-Prime, and the cryptographic guarantees of the FRI-based STARK protocol.

Soundness Properties

Witness binding. The Rescue output row is boundary-asserted to match the public witness commitment. Forging a compliant-looking proof for a non-compliant amount requires finding a second preimage of Rescue-Prime—a problem believed to be computationally infeasible.

Range validity. Binary bit columns satisfy the constraint $b \times (1 - b) = 0$ at every row. Any non-binary value would violate this degree-2 constraint, causing proof generation to fail or the proof to be rejected by the verifier.

Inequality integrity. The subtraction gadget computes limit − amount using 2-limb arithmetic with carry propagation. The final borrow bit is boundary-asserted to be zero, enforcing that no underflow occurred—i.e., the amount does not exceed the limit.

Public input binding. Canonical public inputs are hashed by the verifier and boundary-asserted into the trace at row 0. Any attempt to alter the public inputs after proof generation will cause verification to fail.

SECURITY ASSUMPTION

VES-STARK security is conditional on the collision resistance of Rescue-Prime and the soundness of the FRI protocol. No formal security proof has been published; the security estimate is internal. External review of constraint degree analysis, Rescue parameters, and FRI folding security is recommended before deployment in high-assurance contexts.

Threat Surface & Mitigations

ATTACK VECTOR	DESCRIPTION	MITIGATION
Bit manipulation	Injecting non-binary values into range-check columns	Binary constraint: b(1−b) = 0 at every row
Subtraction forgery	Manipulating the carry chain to fake compliance	Limb-wise equations with final borrow = 0 boundary assertion
Commitment forgery	Committing to a different amount than proven	Rescue output boundary assertion against public commitment
Policy substitution	Replaying a proof against a different policy	Verifier recomputes policy hash; AIR binds policy limit
Public input tampering	Altering event metadata after proof generation	Row-0 boundary assertions on canonical hash
Proof inflation	Submitting oversized proofs to exhaust verifier resources	Proof size limit: 10 MB maximum

Proof Options & Security Levels

DEFAULT PROFILE

Standard

FRI queries	28
Blowup factor	16×
Grinding factor	16
Field extension	None
Est. security	~100-bit

RECOMMENDED

Secure

FRI queries	40
Blowup factor	16×
Grinding factor	20
Field extension	Quadratic
Est. security	~128-bit

DEVELOPMENT ONLY

Fast

FRI queries	20
Blowup factor	16×
Grinding factor	8
Field extension	None
Est. security	~80-bit

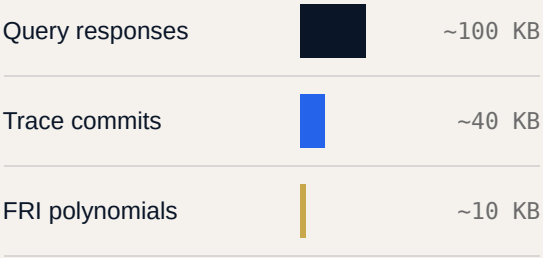
Section IX

Performance Profile

Benchmarked on modern x86-64 hardware with release-optimized builds. Performance reflects the standard proof profile with 28 FRI queries and 16× blowup.

PROOF GENERATION	VERIFICATION	PROOF SIZE	COMMITMENT HASH
500–2000ms	50–200ms	100–200KB	<1ms

Proof Size Breakdown



Proof size scales logarithmically with trace length and linearly with FRI query count. Higher security profiles increase the query count, proportionally increasing proof size.

Comparison: STARKs vs. Alternatives

SYSTEM	PROOF SIZE	VERIFY	SETUP	PQ
VES-STARK	~150 KB	<200ms	None	Yes
Groth16	~0.2 KB	<10ms	Trusted	No
Plonk	~0.5 KB	<20ms	Universal	No
Bulletproofs	~0.7 KB	~500ms	None	No

STARKs trade larger proof size for transparency, post-quantum security, and no trusted setup. For compliance applications where trust assumptions matter more than byte count, this is the correct trade-off.

Section X

Conclusion

Commerce compliance has operated on the assumption that verification requires disclosure. StateSet STARK eliminates that assumption entirely.

By encoding compliance logic as algebraic constraints over the Goldilocks field and proving satisfaction through transparent STARK proofs, the system enables a new category of privacy-preserving commerce infrastructure. Transaction amounts remain encrypted end-to-end. Compliance is verified cryptographically. No trusted third party holds a master key. No quantum computer threatens the validity of historical proofs.

The multi-policy framework supports the three most common compliance patterns in digital commerce today—AML thresholds, order caps, and delegated agent authorization—with a clean extension path for new policy types. Batch proving reduces per-event overhead by two orders of magnitude for high-throughput systems. Native bindings for JavaScript, Python, and Rust meet development teams in their existing toolchains.

The system is production-hardened: API keys are zeroized from memory, proof sizes are bounded, public inputs are canonicalized, and the threat model is explicitly documented. What remains is the work of integration—connecting the proof pipeline to sequencers, blockchain anchoring layers, and the commerce platforms that will benefit from provable compliance without data exposure.

The future of compliance is not more disclosure. It is better mathematics.

TECHNICAL APPENDIX

Constraint Summary

COMPONENT	CONSTRAINTS	MAX DEGREE
Binary bit checks	64	2
Limb recomposition	8	2
Subtraction gadget	4	2
Rescue half-rounds	48	10
Hash I/O binding	12	3–4
Public input binding	8	2
Structural selectors	13	2
Total	**157**	**10**

Trace Layout (Per-Event AIR)

COLUMNS	PURPOSE	WIDTH
0–7	Amount limbs (u8)	8
8–15	Threshold limbs (u8)	8
16–23	Difference limbs	8
24–25	Borrow bits	2
26–57	Amount bit decomposition	32
58–89	Threshold bit decomposition	32
90–101	Rescue-Prime state	12
102–109	Public input binding	8
Total trace width		**96***

*Column indices are illustrative. Actual layout may vary with implementation version.

Glossary

AIR — Algebraic Intermediate Representation. The constraint system that encodes computation as polynomial equations.

FRI — Fast Reed-Solomon Interactive Oracle Proof. The core STARK sub-protocol for proximity testing.

References

1. Ben-Sasson, E., et al. “Scalable, transparent, and post-quantum secure computational integrity.” IACR ePrint, 2018.
2. Aly, A., et al. “Design of Symmetric-Key Primitives for Advanced Cryptographic Protocols.” IACR ToSC, 2020.
3. Winterfell STARK library, v0.10. github.com/facebook/winterfell.
4. RFC 8785: JSON Canonicalization Scheme (JCS). IETF, 2020.
5. Goldilocks field: Plonky2 Technical Reference. Polygon Zero, 2022.

Goldilocks — The 64-bit prime field with $p = 2^{64} - 2^{32} + 1$.

LDE — Low-Degree Extension. Evaluating trace polynomials over a domain larger than the trace length.

Rescue-Prime — An arithmetization-friendly hash function optimized for STARK constraint systems.

Witness — The private input (transaction amount) known only to the prover.

ABOUT STATESET

StateSet builds cryptographic infrastructure for commerce. The VES-STARK protocol is the foundation of StateSet's privacy-preserving compliance layer, enabling verifiable commerce without data exposure.

DOCUMENT INFORMATION

Version 0.2.1 · March 2026
Classification: Public
Protocol: VES-STARK
Framework: Winterfell v0.10